CS402/MA492 Cryptography

Tobias Rossmann
School of Mathematics, Statistics and Applied Mathematics
NUI Galway

Lecture 12:

Cryptography in Python, part 3

Topics

- General affine and Hill ciphers (see A2/Q2).
- Solving modular systems of linear equations.
- Linear feedback shift registers (see A2/Q3–Q4).

Setup

In [1]:

```
from cs402 import *
```

Note. Some of the functions that we'll use below require SymPy (https://www.sympy.org).

General affine and Hill ciphers

Higher-dimensional affine ciphers can be created using `HigherAffineCipher`.

You need to specify both the base alphabet and the dimension.

In [2]:

```
H = HigherAffineCipher(ALPHABET68, 2)
```

Recall: keys are pairs (= tuples) (A, b) consisting of an invertible matrix A (modulo the length of the alphabet) and a vector b.

Matrices are represented using lists of lists (of rows) and vectors as (plain, flat) lists.

For Hill ciphers, just take `b = [0,..., 0]` of the right length.

For example, the matrix $\begin{bmatrix} 1 & 4 \\ 9 & 25 \end{bmatrix}$ is represented by

```
[[1,4],[9,25]]
```

and the column vector $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ by

```
[1,2]
```

In [3]:

```
A = [[1,4],[9,25]]
b = [1,2]
```

You may find the functions `modular_inverse_of_matrix` and `modular_matrix_product` useful:

In [4]:

```
Ainv = modular_inverse_of_matrix(A, 68)
Ainv
```

Out[4]:

```
[[41, 56], [7, 37]]
```

In [5]:

```
modular_matrix_product(A, Ainv, 68)
```

Out[5]:

```
[[1, 0], [0, 1]]
```

In [6]:

```
p = 'Two dimensional affine ciphers need strings of even length'
s = H.encrypt_string((A,b),p)
s
```

Out[6]:

```
' fhkKzDgsz:qoZeZv7MREphIbPyDw9ZLvc;VBBYpM:lUJXpqMtgbC:M:Wi'
```

In [7]:

```
H.decrypt_string((A,b),s)
```

Out[7]:

```
'Two dimensional affine ciphers need strings of even length'
```

Digraph frequencies

Hint. For A2/Q2, use “random English texts” and the function `digraph_ranking` to find the most frequently used digraphs over `ALPHABET27` . To give you an idea, we'll look at this in the case of A1/Q3 next week.

Auxiliary functions

Suppose we wish to find $x, y \in \mathbb{Z}_{68}$ satisfying the system of linear equations

$$\begin{aligned} x + 4y &\equiv 1 \pmod{68} \\ 9x + 25y &\equiv 2 \pmod{68} \end{aligned}$$

In [8]:

```
A = [[1,4],[9,25]]
b = [1,2]
solve_modular_linear_system(A,b,68)
```

Out[8]:

```
[17, 13]
```

In [9]:

```
x,y = 17, 13
```

In [10]:

```
(x + 4 * y) % 68
```

Out[10]:

```
1
```

In [11]:

```
(9*x + 25*y) % 68
```

Out[11]:

```
2
```

Hint. Use this function for known-plaintext attacks on LFSRs (see A2/Q3).

Linear feedback shift registers

Recall: an LFSR is completely determined by its length L and connection polynomial

$$C(X) = 1 + c_1 X + \cdots + c_L X^L.$$

The associated feedback function is

$$f(s_{j-1}, \ldots, s_{j-L}) = c_1 s_{j-1} + \cdots + c_L s_{j-L}.$$

In the Python module `cs402`, in place of the connection polynomial, we provide a list of those indices i with $c_i = 1$.

Example. The LFSR of length 4 with connection polynomial

$$C(X) = 1 + X + X^3 + X^4$$

is defined as follows.

In [12]:

```
L = LFSR(4, [1,3,4])
```

By default, the initial state is zero.

In [13]:

```
L.state
```

Out[13]:

```
[0, 0, 0, 0]
```

Alternatively, we may specify an initial state upon creation of an LFSR:

In [14]:

```
L = LFSR(4, [1,3,4], [1,0,0,0])
```

In [15]:

```
L.state
```

Out[15]:

```
[1, 0, 0, 0]
```

Important. `L.state` corresponds to $(s_{j-L}, \ldots, s_{j-1})$. Note the order. (The state vector is shifted to the *left* here!)

In our example, $s_0 = 1$ and $s_1 = s_2 = s_3 = 0$.

`L.advance()` advances the LFSR to the next state and returns the previous value of `L.state[0]`.

In [16]:

```
for i in range(6):
    print('Output:', L.advance())
    print('New state:', L.state)
```

```
Output: 1
New state: [0, 0, 0, 1]
Output: 0
New state: [0, 0, 1, 1]
Output: 0
New state: [0, 1, 1, 1]
Output: 0
New state: [1, 1, 1, 0]
Output: 1
New state: [1, 1, 0, 0]
Output: 1
New state: [1, 0, 0, 0]
```

We can find the least period associated with a given initial state:

In [17]:

```
L.period([1,0,0,0])
```

Out[17]:

6

Is this best possible? Let's perform a brute-force search:

In [18]:

```
best_state = [0,0,0,0]
max_period = 1
for a in [0,1]:
    for b in [0,1]:
        for c in [0,1]:
            for d in [0,1]:
                period = L.period([a,b,c,d])
                if period > max_period:
                    max_period = period
                    best_state = [a,b,c,d]
```

In [19]:

```
print(max_period)
print(best_state)
```

```
6
[0, 0, 0, 1]
```

Exercise. Write cleaner code for this.

Algebraic interpretation (requires some field theory, group theory, and ring theory):

$$\begin{aligned} C(X) &= 1 + X + X^3 + X^4 \\ &= (X+1)^2(X^2+X+1) \end{aligned}$$

over $\mathbb{Z}_2$. The unit group of

$$\begin{aligned} \mathbb{Z}_2[X]/(C(X)) &\cong \mathbb{Z}_2[X]/((X+1)^2) \oplus \mathbb{Z}_2[X]/(X^2+X+1) \\ &\cong \mathbb{Z}_2[X]/(X^2) \oplus \mathbb{F}_4 \end{aligned}$$

has exponent (https://groupprops.subwiki.org/wiki/Exponent_of_a_group) 6. In fact, it is cyclic of order 6 and generated by the residue class containing X.